Prompt: Implement RuleArena Research Question 2 — Reliability/Autonomy Spectrum Experiments

Role

You are a senior ML research engineer helping implement a systematic ablation study for the **RuleArena benchmark** as part of a multi-benchmark research project. This work must be **structurally consistent** with Aditya Kumar's existing MedCalc-Bench implementation in the `Sylindril/AgentProject` repository (`benchmark/medcalc/` folder), since both benchmarks will appear in the same research paper.

Context

Project Overview

- **Research group:** ~5 PhD students under Professor William Cohen (CMU ML / Google DeepMind), each running the same experimental framework on different benchmarks.
- **Shared repo:** `Sylindril/AgentProject` — Aditya's MedCalc benchmark is the reference implementation.
- **My benchmark:** RuleArena (Zhou et al., ACL 2025) — 816 problems across 3 domains (Airline baggage fees, NBA transactions, Tax regulations), 3 complexity levels (0, 1, 2).
- **My repo:** `Joshh99/ptp-behavior-distillation` — will later be integrated into the shared repo as `benchmark/rulearena/`.
- **Model:** `deepseek-ai/DeepSeek-V3` via Together.ai API (model string: `deepseek-ai/DeepSeek-V3`, uses V3-0324 weights). Single model only for this experiment series.
- **Budget:** ~$30-40 for the full experiment suite.

Research Question 2: Reliability/Autonomy Spectrum

> **What is the optimal point on the reliability-generality-cost curve for production deployment?**

We evaluate multiple architectural "levels" that trade off LLM autonomy against programmatic control, measuring accuracy, cost, latency, and failure modes on the same benchmark problems.

Step 1: Study the Reference Implementation

Before writing any code, thoroughly examine Aditya's MedCalc implementation to understand the conventions you must follow. Explore these files in `Sylindril/AgentProject`:

```
benchmark/medcalc/
  config.py            # ExperimentLevel enum, ExperimentConfig dataclass, ABLATION_CONFIGS dict
  runner.py            # BenchmarkRunner class, run_ablation_study(), generate_report()
  cli.py               # Click-based CLI entry point
  dataset/
    loader.py          # MedCalcDataset class, MedCalcInstance dataclass, stratified_sample()
  experiments/
    README.md          # Layer hierarchy documentation (L0 -> L5C)
    base.py            # BaseExperiment ABC, ExperimentResult dataclass
    baseline.py        # L0: Direct LLM call
    l1_ptool.py        # L1: @ptool structured extraction
    l2_distilled.py    # L2: Python-first with LLM fallback
    l3_react.py        # L3: ReAct autonomous agent
    ...                # (L4, L5 variants)
  metrics/
    aggregator.py      # MetricsAggregator, AggregatedMetrics, CalculatorMetrics, CategoryMetrics
  reports/
    generator.py       # ReportGenerator: HTML report with Plotly charts
    regenerate_report.py # Rebuild report.html from metrics.json without re-running
  response_logger.py   # Save raw prompts/responses for debugging
```

Key patterns to replicate:

1. `ExperimentResult` dataclass — per-instance result with: predicted, expected, correct (bool), time_seconds, cost_usd, input_tokens, output_tokens, error (optional), raw_response, metadata dict.
2. `AggregatedMetrics` dataclass — per-experiment summary with: accuracy (exact + tolerance), total cost, avg latency, token counts, error rate, per-subcategory breakdown.
3. `metrics.json` — the canonical output format: `{experiment_name: AggregatedMetrics.to_dict()}`.
4. `report.html` — self-contained HTML with Plotly charts, comparison tables, per-subcategory heatmaps.
5. `BenchmarkRunner.run_ablation_study()` — iterates over experiment configs, runs each, aggregates metrics, generates report.
6. Default model in MedCalc config is `"deepseek-v3-0324"` — confirms alignment with our model choice.

Study Aditya's code carefully before proceeding. Note the exact field names in `ExperimentResult` and `AggregatedMetrics` — our output JSON must use the same schema so that the final paper's analysis scripts work across benchmarks.

Step 2: Define the RuleArena Experiment Levels

Map our research question's levels to concrete implementations. These are the levels for RuleArena (note: fewer than MedCalc's L0-L5C, because RuleArena's domain is different):

L0: Pure Python Baseline (No LLM)

- **Architecture:** Hardcoded Python rule engine. No LLM calls at all.
- **Input:** Oracle-extracted parameters (from ground truth annotations or manually parsed).
- **Process:** `calculate(ground_truth_params)` using the reference implementation from RuleArena's codebase.
- **Purpose:** Establishes the ceiling for deterministic computation accuracy and measures coverage — what percentage of problems can be solved without any LLM?
- **Expected:** ~100% accuracy on covered cases, but limited coverage (~60-70%) due to edge cases.
- **Cost:** $0.

L0F: Chain-of-Thought Baseline (Pure LLM, No Structure)

- **Architecture:** Direct LLM call with all rules in the prompt + CoT reasoning. Replicates the original RuleArena paper's evaluation methodology.
- **Input:** Full problem text + domain rules (as provided in RuleArena).
- **Process:** Single LLM call with "Think step-by-step" instruction. LLM reasons through rules and computes answer end-to-end.
- **Purpose:** Establishes the pure-LLM baseline (no tools, no structure). This is the "what if we just throw the problem at the LLM?" condition.
- **Expected:** 0-20% accuracy (consistent with RuleArena paper's findings for non-reasoning models).
- **Cost:** $ (single LLM call per problem).

L1: PTool Extraction + Python Calculation (Structured Workflow)

- **Architecture:** LLM extracts structured parameters → Python deterministic calculation.
- **Input:** Problem text + rules + JSON schema for expected parameters.
- **Process:** (1) LLM extracts parameters as JSON, (2) Python `calculate()` function computes answer from extracted params.
- **Purpose:** The core PTP hypothesis — separation of extraction (LLM) from computation (Python) improves reliability.
- **Expected:** 80-95% accuracy depending on extraction quality.
- **Cost:** $ (single LLM call for extraction).

L1-TA: Tool-Augmented Baseline (LLM Writes + Executes Code)

- **Architecture:** LLM generates Python code, then code is executed to produce the answer.
- **Input:** Problem text + rules + instruction to write executable Python code.
- **Process:** (1) LLM generates code that encodes rule logic + computation, (2) Code is executed in sandbox, (3) Output is parsed as answer.
- **Purpose:** Tests whether LLM-generated code is more reliable than LLM-generated reasoning (CoT). This is the "tool augmentation" condition from the RuleArena paper.
- **Expected:** 30-70% accuracy (higher than CoT, lower than L1 due to code generation errors).
- **Cost:** $ (single LLM call + code execution).

L3: ReAct Agent (Autonomous Multi-Step)

- **Architecture:** Full ReAct agent with Think/Act/Observe loop and access to tools.
- **Input:** Problem text + rules + available tools (parameter extractors, rule lookups, calculators).
- **Process:** Agent autonomously decides which tools to call, iterates until confident in answer.
- **Tools available:** `extract_passenger_info()`, `lookup_baggage_rules()`, `calculate_fee()`, etc.
- **Purpose:** Tests whether autonomous agent flexibility improves over structured L1 workflow.
- **Expected:** 40-60% accuracy, 5-10x cost vs L1, high variance.
- **Cost:** $$$$$ (multiple LLM calls per problem).

(Optional) L2: Router + Multiple Specialized L1s

- **Architecture:** LLM classifier routes to domain-specific L1 ptools.
- **Only implement if time permits after L0, L0F, L1, L1-TA, L3 are complete.**

Step 3: Implementation Architecture

Create the following structure in my repository, mirroring MedCalc's conventions:

```
benchmark/rulearena/
  __init__.py
  config.py                  # ExperimentLevel enum, ExperimentConfig, ABLATION_CONFIGS
  runner.py                  # BenchmarkRunner (mirrors medcalc/runner.py)
  cli.py                     # CLI entry point
  dataset/
    __init__.py
    loader.py                # RuleArenaDataset, RuleArenaInstance dataclass
  experiments/
    __init__.py
    README.md                # Level hierarchy for RuleArena
    base.py                  # BaseExperiment, ExperimentResult (SAME schema as MedCalc)
    l0_python.py             # L0: Pure Python baseline
    l0f_cot.py               # L0F: Chain-of-Thought baseline
    l1_ptool.py              # L1: PTool extraction + Python calc
    l1ta_tool_augmented.py   # L1-TA: LLM code generation + execution
    l3_react.py              # L3: ReAct agent
  metrics/
    __init__.py
    aggregator.py            # MetricsAggregator, AggregatedMetrics (SAME schema as MedCalc)
  reports/
    __init__.py
    generator.py             # ReportGenerator (produces report.html + metrics.json)
    regenerate_report.py     # Rebuild from metrics.json
  calculators/               # Domain-specific Python calculation engines
    __init__.py
    airline.py               # Airline baggage fee calculator
    nba.py                   # NBA transaction calculator
    tax.py                   # Tax calculation engine
  rules/                     # Rule files (loaded from RuleArena dataset)
    __init__.py
  response_logger.py         # Raw prompt/response logging
```

Step 4: Critical Implementation Details

4.1 Dataset Loader

```python
```

```python
@dataclass
class RuleArenaInstance:
    """Single benchmark instance. Mirrors MedCalcInstance structure."""
    instance_id: str              # Unique identifier
    domain: str                   # "airline" | "nba" | "tax"
    complexity_level: int         # 0, 1, 2
    problem_text: str             # The natural language question
    rules_text: str               # Domain rules provided to the model
    ground_truth_answer: Any      # Expected numeric/string answer
    ground_truth_explanation: str # Step-by-step solution (if available)
    metadata: dict                # Additional fields from RuleArena JSON
```

- Load from `external/RuleArena/` (cloned from `SkyRiver-2000/RuleArena` GitHub repo).
- Implement `stratified_sample(n, seed)` that samples proportionally across domains and complexity levels.
- Support both full dataset and debug mode (first N instances).

4.2 ExperimentResult (MUST match MedCalc schema)

```python
@dataclass
class ExperimentResult:
    """Per-instance result. Schema MUST match MedCalc's ExperimentResult."""
    instance_id: str
    predicted: Any                # Model's answer
    expected: Any                 # Ground truth
    correct: bool                 # Whether predicted matches expected
    time_seconds: float           # Wall-clock time for this instance
    cost_usd: float               # API cost for this instance
    input_tokens: int
    output_tokens: int
    error: Optional[str] = None   # Error message if failed
    raw_response: Optional[str] = None
    metadata: dict = field(default_factory=dict)
    # metadata should include: domain, complexity_level, extraction_json (for L1),
    # trace (for L3), code_generated (for L1-TA)
```

4.3 AggregatedMetrics (MUST match MedCalc schema)

The `AggregatedMetrics` must include:

- `experiment_name`, `experiment_level`

- `total_instances`, `correct_exact`, `accuracy_exact`
- `correct_tolerance`, `accuracy_tolerance` (for RuleArena: use +-1% tolerance on numeric answers)
- `total_cost_usd`, `avg_cost_usd`
- `avg_latency_ms`
- `total_input_tokens`, `total_output_tokens`, `total_tokens`
- `error_count`, `error_rate`
- `by_category` dict: keyed by domain ("airline", "nba", "tax") with CategoryMetrics
- `by_calculator` dict: keyed by `"{domain}_level_{complexity}"` with per-subcategory metrics

4.4 Model Configuration

```python
# In config.py
MODEL_CONFIG = {
    "model_id": "deepseek-ai/DeepSeek-V3",
    "provider": "together",
    "api_base": "https://api.together.xyz/v1",
    "pricing": {
        "input_per_million": 0.30,   # $0.30/M input tokens
        "output_per_million": 0.90,  # $0.90/M output tokens
    },
    "default_params": {
        "temperature": 0.0,          # Deterministic for reproducibility
        "max_tokens": 4096,
        "seed": 42,
    }
}
```

4.5 Answer Comparison

RuleArena answers are numeric. Implement comparison logic:

- **Exact match:** `predicted == expected` (after rounding to same precision)
- **Tolerance match:** `abs(predicted - expected) / max(abs(expected), 1e-9) <= 0.01` (1% relative tolerance)
- Parse LLM outputs robustly: extract numbers from text, handle currency symbols, commas, etc.
- Log parsing failures as errors in ExperimentResult.

4.6 Cost Tracking

Track per-instance cost using Together.ai's response headers or token counts:

```python
cost = (input_tokens * 0.30 / 1_000_000) + (output_tokens * 0.90 / 1_000_000)
```

Step 5: Experiment-Specific Implementation Notes

L0 (Pure Python)

- Use RuleArena's reference calculator code (in their repo under evaluation/).
- For oracle parameters: parse from the ground_truth_explanation field, or use the dataset's structured annotations if available.
- If oracle params aren't directly available, manually create a mapping for a subset and measure coverage.

L0F (Chain-of-Thought)

- Prompt template:

```
You are solving a {domain} calculation problem.

RULES:
{rules_text}

PROBLEM:
{problem_text}

Think step by step. Show your reasoning, then give your final numeric answer on the last line as:
ANSWER: <number>
```

- Parse the "ANSWER: " line from the response.
- 0-shot only (no examples).

L1 (PTool Extraction)

- Two-phase approach:
  1. **Extraction prompt:** Ask LLM to extract structured parameters as JSON based on a predefined schema for each domain.
  2. **Calculation:** Pass extracted JSON to Python calculator.
- Define JSON schemas per domain (airline params differ from NBA params differ from tax params).

- Log the extracted JSON in `metadata["extraction_json"]` for error analysis.
- Example airline extraction schema:

```
json

{
  "num_bags": int,
  "bags": [
    {
      "weight_lbs": float,
      "length_inches": float,
      "width_inches": float,
      "height_inches": float
    }
  ],
  "route_type": "domestic" | "international",
  "frequent_flyer_status": str,
  "airline": str
}
```

L1-TA (Tool-Augmented)

- Prompt the LLM to write executable Python code that solves the problem.
- Execute in a sandboxed environment (use `exec()` with restricted globals or subprocess).
- Capture stdout as the answer.
- Log generated code in `metadata["code_generated"]`.
- Include defensive prompt instructions: "Include error handling. Print only the final numeric answer."

L3 (ReAct Agent)

- Implement a Think/Act/Observe loop with max 10 steps.
- Available tools vary by domain:
  - **Airline:** `extract_passenger_info(text) -> JSON`, `lookup_fee_table(airline, route_type) -> table`, `calculate_baggage_fee(params) -> number`
  - **NBA:** `extract_team_info(text) -> JSON`, `lookup_salary_rules(scenario) -> rules`, `calculate_transaction(params) -> result`
  - **Tax:** `extract_taxpayer_info(text) -> JSON`, `lookup_tax_brackets(year, status) -> brackets`, `calculate_tax(params) -> number`
- Log full agent trace in `metadata["trace"]`.
- Track number of LLM calls (steps) per problem.

Step 6: Running Experiments

Execution Order

1. **L0F (CoT baseline)** — cheapest, establishes LLM-only floor
2. **L1 (PTool)** — core hypothesis test
3. **L1-TA (Tool-Augmented)** — comparison condition
4. **L3 (ReAct)** — most expensive, run last
5. **L0 (Pure Python)** — no API calls, can run anytime

Run Configuration

```python
# Full run: all 816 problems (300 airline + 216 NBA + 300 tax)
# Debug run: 30 problems (10 per domain, stratified by complexity)
# Estimated costs (full run):
#   L0F: ~$1-2 (816 single calls)
#   L1:  ~$1-2 (816 single calls)
#   L1-TA: ~$2-3 (816 single calls, longer outputs)
#   L3:  ~$15-25 (816 * ~5 calls avg)
#   Total: ~$20-32
```

CLI Interface (mirror MedCalc's cli.py)

```bash
# Run specific experiments
python -m benchmark.rulearena run --experiments l0f_cot l1_ptool --debug

# Run all experiments
python -m benchmark.rulearena run --experiments all --seed 42

# List available experiments
python -m benchmark.rulearena list

# Generate report from existing metrics.json
python -m benchmark.rulearena report --results-dir ./benchmark_results/rulearena
```

Step 7: Output Format Requirements

metrics.json (Critical — must match MedCalc's schema)

```json
{
  "l0f_cot": {
    "experiment_name": "l0f_cot",
    "experiment_level": "L0F",
    "total_instances": 816,
    "correct_exact": 45,
    "accuracy_exact": 0.055,
    "correct_tolerance": 52,
    "accuracy_tolerance": 0.064,
    "total_cost_usd": 1.23,
    "avg_cost_usd": 0.0015,
    "avg_latency_ms": 2340,
    "total_input_tokens": 1250000,
    "total_output_tokens": 410000,
    "total_tokens": 1660000,
    "error_count": 12,
    "error_rate": 0.015,
    "by_category": {
      "airline": { "category": "airline", "total_instances": 300, "accuracy_exact": 0.04, ... },
      "nba": { ... },
      "tax": { ... }
    },
    "by_calculator": {
      "airline_level_0": { "calculator_name": "airline_level_0", "total_instances": 100, ... },
      "airline_level_1": { ... },
      "airline_level_2": { ... },
      "nba_level_0": { ... },
      ...
    }
  },
  "l1_ptool": { ... },
  "l1ta_tool_augmented": { ... },
  "l3_react": { ... },
  "l0_python": { ... }
}
```

report.html Requirements

- Self-contained HTML with embedded Plotly.js charts
- Sections: Executive Summary, Experiment Comparison table, Accuracy by Level chart, Cost vs Accuracy scatter, Per-Domain Breakdown, Per-Complexity heatmap
- Same visual style as MedCalc's report (use the same CSS/chart patterns from `medcalc/reports/generator.py`)
- Include experiment metadata: model name, timestamp, seed, total cost

Per-Instance Results (detailed_results.json)

```json
{
  "l1_ptool": [
    {
      "instance_id": "airline_000",
      "domain": "airline",
      "complexity_level": 0,
      "predicted": 1245.0,
      "expected": 1245.0,
      "correct": true,
      "time_seconds": 2.34,
      "cost_usd": 0.0015,
      "input_tokens": 1523,
      "output_tokens": 187,
      "error": null,
      "metadata": {
        "extraction_json": {"num_bags": 3, "bags": [...], ...}
      }
    },
    ...
  ]
}
```

Step 8: Implementation Priority

Implement in this order (each phase should be testable independently):

1. **Phase 1: Infrastructure** — config.py, dataset/loader.py, experiments/base.py, metrics/aggregator.py
2. **Phase 2: L0F (CoT) experiment** — simplest to implement, validates the pipeline end-to-end

3. **Phase 3: L1 (PTool) experiment** — the core experiment, requires domain-specific extraction schemas + calculators
4. **Phase 4: Report generator** — produce report.html + metrics.json after L0F and L1 are working
5. **Phase 5: L1-TA + L3** — remaining experiments
6. **Phase 6: L0 baseline** — requires oracle parameter extraction, can be done independently
7. **Phase 7: CLI + polish** — cli.py, README.md, integration testing

Start with Phase 1. After each phase, run a debug test (N=10) to validate before proceeding.

Constraints and Reminders

- **Single model only:** All experiments use `deepseek-ai/DeepSeek-V3`. Do NOT add multi-model support.
- **Seed everything:** Use `seed=42` for all LLM calls and any random sampling.
- **Temperature 0:** All experiments use temperature=0 for deterministic outputs.
- **Log everything:** Save raw prompts and responses for every instance (for debugging and paper supplementary).
- **Error handling:** Never crash on a single instance failure. Log the error, record it in ExperimentResult, and continue.
- **Caching:** Use cachier or similar to avoid re-running expensive API calls during development. Cache key should include: model, prompt hash, temperature, seed.
- **No emoticons in code.** This is academic research code.
- **Match MedCalc's field names exactly** in ExperimentResult and AggregatedMetrics. The final paper's analysis scripts will iterate over `metrics.json` files from all benchmarks.
- **RuleArena data:** Clone from `https://github.com/SkyRiver-2000/RuleArena` into `external/RuleArena/`. Do NOT modify the external data.
- **Attribution:** Include RuleArena paper citation in README and any generated reports.

Verification Checklist

Before considering any phase complete, verify:

- [ ] `metrics.json` schema matches MedCalc's schema (diff field names)
- [ ] `report.html` loads in browser with working Plotly charts
- [ ] Debug run (N=10) completes without errors for each experiment
- [ ] Cost tracking matches expected ranges (sanity check against Together.ai dashboard)

- [ ] Per-domain and per-complexity breakdowns are populated correctly
- [ ] All raw responses are logged and retrievable
- [ ] `regenerate_report.py` can rebuild report.html from metrics.json alone